

Quick Review

meta/tasks.parquet



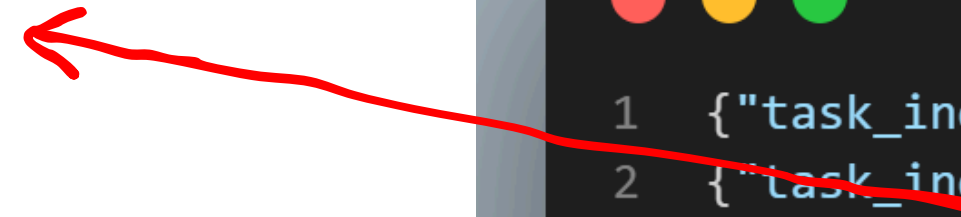
```
1 {"task_index":0,"__index_level_0__":"fold the garment on the table"}
```



```
1 {"task_index":0,"__index_level_0__":"use both arms to smooth out and fold the long pants on the table"}
2 {"task_index":1,"__index_level_0__":"use both arms to smooth out and fold the short pants on the table"}
3 {"task_index":2,"__index_level_0__":"use both arms to smooth out and fold the long-sleeve top on the table"}
4 {"task_index":3,"__index_level_0__":"use both arms to smooth out and fold the short-sleeve top on the table"}
5
```

data/chunk-000/file-000.parquet

```
{  
  "observation.state" : [...],  
  "action" : [...],  
  :  
  "task_index" : 0  
}
```



```
1 {"task_index":0,"__index_level_0__":"use both arms to smo  
2 {"task_index":1,"__index_level_0__":"use both arms to smo  
3 {"task_index":2,"__index_level_0__":"use both arms to smo  
4 {"task_index":3,"__index_level_0__":"use both arms to smo  
5
```

meta/info.json

```
{
  "codebase_version": "v3.0",
  "robot_type": null,
  "total_episodes": 1000,
  "total_frames": 265798,
  ✨ "total_tasks": 4,
    "chunks_size": 1000,
  "data_files_size_in_mb": 100,
  "video_files_size_in_mb": 200,
  "fps": 30,
  "splits": {
    "train": "0:1000"
  },
  "data_path": "data/chunk-{chunk_index:03d}/file-{file_index:03d}",
  "video_path": "videos/{video_key}/chunk-{chunk_index:03d}/video-{video_index:03d}.mp4",
  "features": {
    "observation.state": {
      "dtype": "float32",
      "shape": [
        12
      ],
    },
  },
}
```

Top_short

Before

```
=====  
Total Episodes: 60  
Average Return: 190.89 ± 64.80  
Success Rate: 11.67%  
=====
```

After

```
=====  
Total Episodes: 60  
Average Return: 128.04 ± 49.29  
Success Rate: 1.67%  
=====
```


Can't believe it

Top_long

Before

```
=====  
Total Episodes: 60  
Average Return: 135.71 ± 46.86  
Success Rate: 68.33%  
=====
```

After

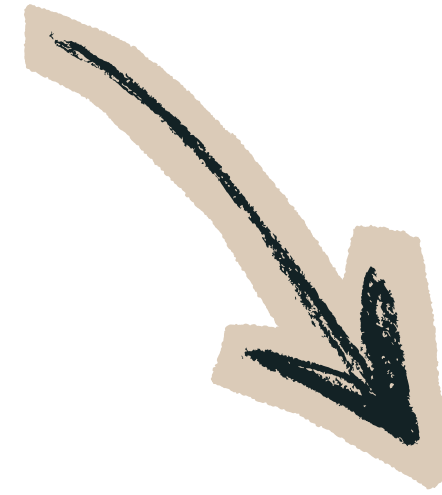
```
=====  
Total Episodes: 60  
Average Return: 113.25 ± 8.53  
Success Rate: 0.00%  
=====
```

What's going on?

lehome-challenge/.venv/lib/python3.11/site-packages/lerobot/policies/smolvla/configuration_smolvla.py



```
1 # Finetuning settings
2 freeze_vision_encoder: bool = True
3 train_expert_only: bool = True
4 train_state_proj: bool = True
```



```
1 policy:
2     type: smolvla
3     train_expert_only: false
```

Community datasets



SmoIVLA

Affordable robots

LeRobot



`train_expert_only=true`

$[a_t, a_{t+1}, \dots, a_{t+H}]$

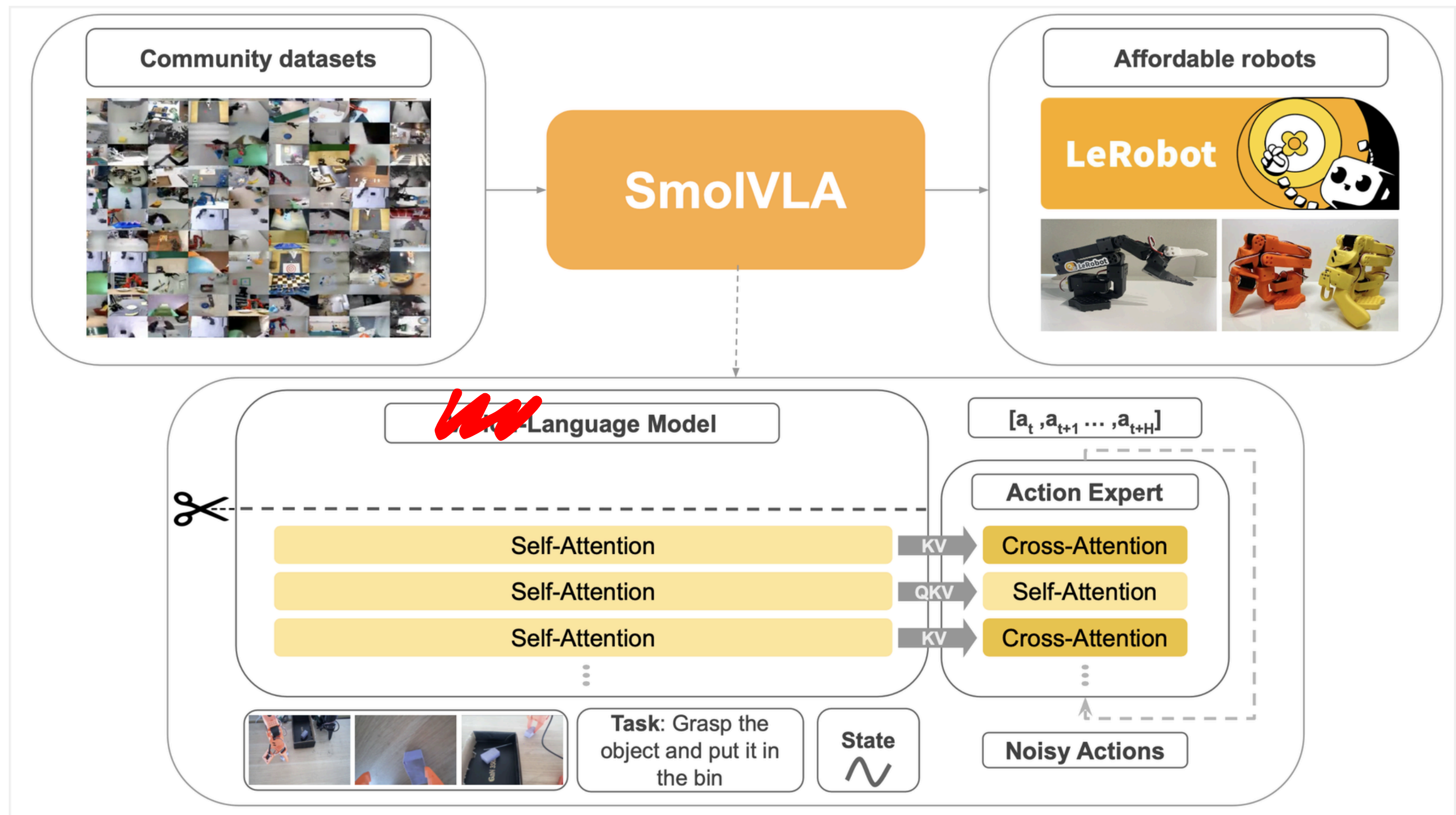
Action Expert

Cross-Attention

Self-Attention

Cross-Attention

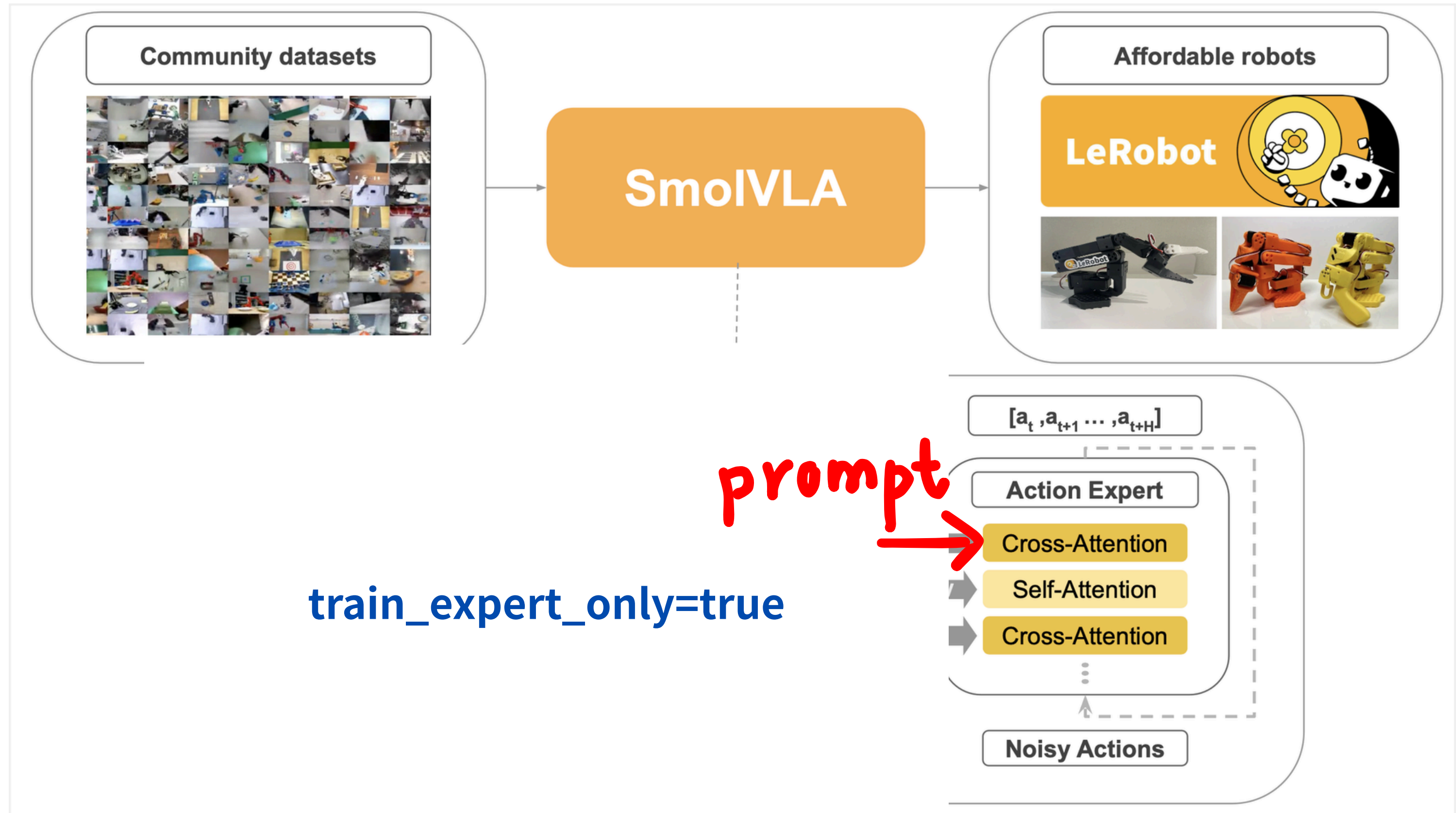
Noisy Actions



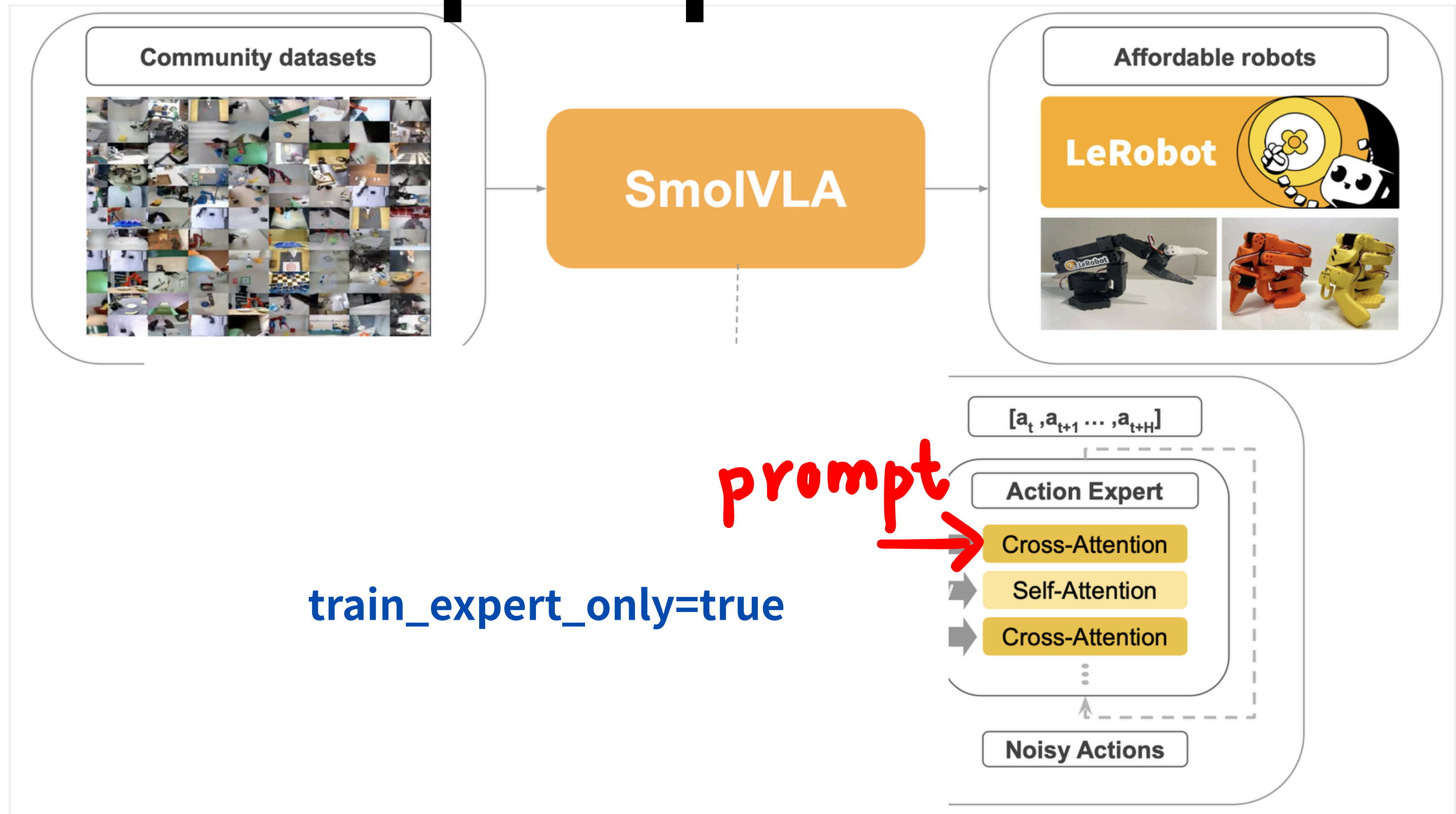
train_expert_only=false

**Why baseline is ok with train_expert_only,
but prompt optimization will collapse?**

Baseline



Prompt Optimization



lehome-challenge/.venv/lib/python3.11/site-
packages/lerobot/policies/smolvla/smolvlm_wi
th_expert.py

```
1  def set_requires_grad(self):
2      if self.freeze_vision_encoder:
3          self.get_vlm_model().vision_model.eval()
4          for params in self.get_vlm_model().vision_model.parameters():
5              params.requires_grad = False
6      if self.train_expert_only:
7          self.vlm.eval()
8          for params in self.vlm.parameters():
9              params.requires_grad = False
10     else:
11         # To avoid unused params issue with distributed training
12         last_layers = [self.num_vlm_layers - 1]
13         if (
14             self.num_vlm_layers != self.num_expert_layers
15             and self.num_vlm_layers % self.num_expert_layers == 0
16         ):
17             last_layers.append(self.num_vlm_layers - 2)
18         frozen_layers = [
19             "lm_head",
20             "text_model.model.norm.weight",
21         ]
22         for layer in last_layers:
23             frozen_layers.append(f"text_model.model.layers.{layer}.")
24
25         for name, params in self.vlm.named_parameters():
26             if any(k in name for k in frozen_layers):
27                 params.requires_grad = False
28     # To avoid unused params issue with distributed training
29     for name, params in self.lm_expert.named_parameters():
30         if "lm_head" in name:
31             params.requires_grad = False
32
```

**So, after all that, what's the
result?**


```
=====
Total Episodes: 60
Average Return: 110.13 ± 13.00
Success Rate: 0.00%
=====
```

```
=====
Per-Garment Summary
=====
```

```
Top_Long_Seen_0: Success Rate = 0.00%, Avg Return = 107.69
Top_Long_Seen_1: Success Rate = 0.00%, Avg Return = 119.24
Top_Long_Seen_2: Success Rate = 0.00%, Avg Return = 86.08
Top_Long_Seen_3: Success Rate = 0.00%, Avg Return = 106.94
Top_Long_Seen_4: Success Rate = 0.00%, Avg Return = 121.19
Top_Long_Seen_5: Success Rate = 0.00%, Avg Return = 110.03
Top_Long_Seen_6: Success Rate = 0.00%, Avg Return = 110.09
Top_Long_Seen_7: Success Rate = 0.00%, Avg Return = 107.02
Top_Long_Seen_8: Success Rate = 0.00%, Avg Return = 114.81
Top_Long_Seen_9: Success Rate = 0.00%, Avg Return = 122.42
Top_Long_Unseen_0: Success Rate = 0.00%, Avg Return = 111.01
Top_Long_Unseen_1: Success Rate = 0.00%, Avg Return = 104.98
=====
```

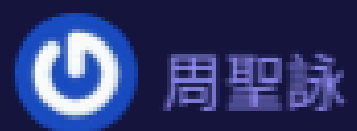


What's next?

- **freeze_vision_encoder: true → false**
- **Increase steps**
- **Reduce lr**
- **Apply KG-Dagger in SmolVLA** 
- **try Gigabrain to generate synthetic data**
- **Synthetic data**
- **~~Apply LoRA on baseline SmolVLA~~**



synthetic data



synthetic data



synthetic data

What is Synthetic Data? No, It's Not "Fake" Data



Learning Keypoints for Robotic Cloth Manipulation using Synthetic Data

What does this paper do?

1. Generate synthetic data
2. Training Keypoint detector

How? (Method)

Using the `generate_flat_meshes.py` script, automatically generate flat meshes of clothing such as T-shirts, shorts, and towels in Blender through programming.

Procedural generation



Flatten fabric mesh (including key point annotations)



Physical simulation deformation (PyFlex / Blender)



Deformed mesh (including updated key point positions)



Constructing a Blender scene (adding obstacles, HDR lighting, materials)



Render images + automatically generate COCO format annotations



Used for training keypoint detection models (Keypoint Detector)

README MIT license

Understand Anything

Turn any codebase into an interactive knowledge graph you can explore, search, and ask questions about.

[English](#) | [中文](#)

[QUICK START](#) [LICENSE](#) [MIT](#) [CLAUDE CODE](#) [PLUGIN](#) [HOMEPAGE](#)



Turn any codebase into an interactive knowledge graph

it takes any codebase and turns it into something you can actually understand:

- visual graph of the system
- clear explanations in plain english
- ask questions like "how does this work?"

you find something interesting on github. but you have no idea what it actually does. the code is big, docs are not that clear, no clear...

you find something interesting on github. but you have no idea what it actually does. the code is big, docs are not that clear, no clear entry point. so you just... skip it, or you save it and you never come back to it. i feel you. this is exactly why tools like...

[in](#) LinkedIn

Robbyant/lingbot-vla: A **Pragmatic VLA Foundation** **Model**